The `QuerySet` object is a lazy object. That descriptor isn't meant to be rude—*lazy* means it waits as long as possible to be evaluated. You might have hundreds or thousands of musicians in your database. Running `.all()` could result in a large amount of data coming back. The `QuerySet` object wraps the query but doesn't execute it immediately. You can further refine your query by performing operations on a `QuerySet`. Accessing an item in the `QuerySet` is what causes it to be evaluated. In the Django shell, running `.all()` directly casts the result as a string, invoking the evaluation.

Achieving laziness

When you call a method on a query manager, it returns a `QuerySet` object that represents an SQL statement that has not yet been run on the database. This can take a little getting used to. Read that again: calling a query method doesn't actually query the database. This is named *lazy execution*, which means the database call doesn't happen until you actually need the data.

Although this can be a little strange to wrap your head around, it's a good thing. It means you can refine a query and build it up over multiple lines of code, and it won't fire until you actually need it to. This is much more efficient than running multiple queries.

In the previous example, when you ran `Musician.objects.all()`, Django created a `QuerySet` object in response. When you access the result by printing it out, converting it to a list, or accessing one of its members, Django then calls the database.

By contrast, you could run the same code, `Musicians.objects.all()`, and then call `.first()`. This is another query method, which returns the first item in a query set. Django is smart enough to optimize the underlying call: rather than fetching all the data and then giving you the first item, it only asks the database for the first item. This is far more efficient.

Likewise, you could call `.count()`, which returns the number of items in a `QuerySet`. Note that this isn't like calling `len()`. Turning the result into a list and determining the length of the list could take a lot of processing, especially if there is a lot of data. The `.count()` query method happens on the database side, and databases are good at counting results. By calling `.count()` on a `QuerySet`, the only data coming back to Django is the number of items in the `QuerySet`, not the values being counted.

Both `.first()` and `.count()` cause the query to be executed as they return data, but not all query methods do so. You can chain calls to `.filter()`, narrowing down the resulting data as you go. Each call adds more to the underlying SQL statement, and it isn't until you perform an action that requires data to be returned that the statement gets executed.

Databases are optimized for searching and filtering on items. As much as possible, you want the database to do the work. You can ask for all musicians and search the result for those with first name *John*, but this is inefficient. Not only do you have to write the search code yourself, but if the database you are using is on the network, all the data